

TAREA 02-B — CORRECCIÓN: EL DINERO

Corta. 27 de agosto de 2026, 22:30 **Corrige la TAREA-02, que por lo demás quedó bien.**

LO QUE SÍ QUEDÓ BIEN — verificado, no asumido

Bytes nulos ninguno Llaves balanceadas los 26 archivos
Compila (esbuild) 125 kb 27 pruebas unitarias existen de verdad
grep de estilos en paginas/ ... vacío Colores en piezas/ y amazon/ . vacío, todo por tokens
Media queries almacén 3 · piezas 2 · páginas 2 Tabla → tarjetas bajo
768px ... sí

Esta vez no hubo corrupción ni reporte inflado. El almacén, las piezas y los tokens se quedan como están.

EL PROBLEMA

El dinero está guardado como texto con decimales

```
// src/datos-falsos/adminEjemplo.js
montoEsperado: '150.00',
totalAPagar:   '42,380.00',

// src/datos-falsos/comisionesEjemplo.js
bonoResidual:  '80.40',
ganaste:       '72.00',
```

La constitución dice enteros en céntimos. *Artículo II — Integridad del dinero.*

S/. 150.00	→	15000
S/. 42,380.00	→	4238000
S/. 80.40	→	8040

Y el formateo está escrito a mano en cada página

```
// P11PanelSocio.jsx:119
valor={`S/. ${socioMaria.billeteraDisponible}`}
```

```
// P14MisComisiones.jsx:25
render: (f) => `S/. ${f.ganaste}`

// P23BandejaConfirmacion.jsx:165
<span>S/. {pedidoSeleccionado.montoEsperado}</span>
```

Ese patrón está repetido en 4 páginas y unas 15 veces.

Por qué importa ahora y no después

Hoy no rompe nada porque nada se calcula. Pero la TAREA-01 va a traer los datos reales de Supabase, **y ahí el dinero viene como entero en céntimos.**

Cuando eso pase:

```
S/. ${15000} → "S/. 15000"    muestra quince mil en vez de ciento cincuenta
'150.00' + '80.40' → "150.0080.40" concatena texto en vez de sumar
parseFloat('1,200.00') → 1     la coma lo corta
```

Arreglarlo ahora son 4 archivos de datos y una función. Después son 20 pantallas.

BLOQUE A · LOS DATOS EN CÉNTIMOS

Convierte todo monto de src/datos-falsos/ a entero en céntimos, y renombra el campo para que se note.

```
// antes
montoEsperado: '150.00',
totalAPagar: '42,380.00',
ganaste: '72.00',

// después
montoEsperadoCent: 15000,
totalAPagarCent: 4238000,
ganasteCent: 7200,
```

La regla de nombres

Todo campo de dinero termina en Cent y es un number entero.

Es a propósito. Si el nombre lo dice, nadie lo suma como texto por descuido, y si mañana alguien escribe `montoCent: '150.00'` se ve raro de inmediato.

Los archivos a tocar

src/datos-falsos/adminEjemplo.js	montos, comisiones, totales del cierre
src/datos-falsos/comisionesEjemplo.js	bonos, desglose, caso cero
src/datos-falsos/socioEjemplo.js	billetera, bono de rango, estimada

❑ Lo que NO se convierte

Los puntos no son dinero. Siguen siendo enteros normales, sin céntimos.

```
puntos: 180,           ❑ se queda igual
puntosGrupales: 1240, ❑ se queda igual
```

Los porcentajes tampoco. porcentaje: '40%' es una etiqueta de texto.

❑ Y de paso: los precios de pack están MAL

Encontrado el 28/08 comparando Kit.jsx contra la base de datos.

```
// Kit.jsx líneas 280-283 – lo que dice hoy
'Kit Emprendedor (S/. 120)'      ❑ correcto
'Pack Ejecutivo (S/. 400)'       ❑ son S/. 360
'Pack Gold (S/. 1,200)'         ❑ correcto
'Pack Empresarial (S/. 2,400)'   ❑ son S/. 8,000
```

Y falta el Pack Familiar de S/. 4,000, que sí existe en la base — es el que Máximo confirmó por WhatsApp: "EL PACK DE 4000 PARA ARMAR SU PACK AL 50%".

Los cinco packs, verificados en Postgres:

EMPRENDEDOR	12000 cent	S/. 120
EJECUTIVO	36000 cent	S/. 360
GOLD	120000 cent	S/. 1,200
FAMILIAR	400000 cent	S/. 4,000
EMPRESARIAL	800000 cent	S/. 8,000

No los escribas a mano otra vez. Léelos de la tabla pack de Supabase. Si mañana cambia un precio, la pantalla lo sigue sola.

Commit.

BLOQUE B · UNA SOLA FUNCIÓN DE FORMATO

Crea src/utilidades/dinero.js:

```

/**
 * Convierte céntimos enteros a texto en soles.
 * 15000 → "S/. 150.00"
 * 4238000 → "S/. 42,380.00"
 * 0 → "S/. 0.00"
 */
export function formatearSoles(centimos) { ... }

/** Solo el número, sin el símbolo. Para tablas apretadas. */
export function formatearMonto(centimos) { ... }

```

Requisitos

- ❑ Usa Intl.NumberFormat con 'es-PE' y moneda PEN
- ❑ Siempre 2 decimales, incluso en montos redondos
- ❑ Separador de miles
- ❑ El cero se muestra "S/. 0.00", NO vacío ni guion
- ❑ Si recibe null o undefined devuelve "S/. 0.00", no "NaN"

Lo del cero no es un detalle. La pantalla P-14 tiene que mostrar "S/. 0.00" bien claro cuando el socio no cobró. Un guion o un espacio en blanco deja al socio sin saber si es cero o si el sistema falló.

Commit.

BLOQUE C · USARLA EN TODAS LAS PANTALLAS

Reemplaza los ~15 lugares donde hoy se arma el texto a mano.

```

// antes
valor={`S/. ${socioMaria.billeteraDisponible}`}

// después
valor={formatearSoles(socioMaria.billeteraDisponibleCent)}

```

Archivos

```

src/paginas/P11PanelSocio.jsx
src/paginas/P14MisComisiones.jsx
src/paginas/P23BandejaConfirmacion.jsx
src/paginas/P25CierreCiclo.jsx
src/paginas/Kit.jsx

```

La regla que queda para siempre

Ninguna pantalla escribe S/. nunca más.

Si aparece el símbolo de soles en un archivo de `src/paginas/`, está mal — salvo en un texto fijo que no sea un monto, como *"1 punto = S/. 1.00"*.

Verificación

```
# los montos ya no se arman a mano
grep -rn 'S/\. \${' src/paginas/      # VACÍO
grep -rn 'S/\. {' src/paginas/       # VACÍO
```

Commit.

BLOQUE D · PRUEBAS

```
Vitest
❑ formatearSoles(15000)    === "S/. 150.00"
❑ formatearSoles(4238000) === "S/. 42,380.00"
❑ formatearSoles(0)       === "S/. 0.00"
❑ formatearSoles(null)    === "S/. 0.00"    no "NaN"
❑ formatearSoles(8040)    === "S/. 80.40"
❑ Ningún monto de datos-falsos es string
❑ Ningún monto de datos-falsos tiene decimales
❑ P-14 en cero sigue mostrando "S/. 0.00" y su explicación
```

La comprobación de que no quedan strings:

```
// recorre los datos falsos y falla si algún campo *Cent no es entero
Object.entries(datos).forEach(([k, v]) => {
  if (k.endsWith('Cent')) {
    expect(Number.isInteger(v)).toBe(true);
  }
});
```

Las 27 pruebas existentes tienen que seguir en verde.

Commit.

ORDEN

```
0º git add -A && git commit -m "estado antes de correccion dinero"
1º Bloque A · datos en céntimos          commit
2º Bloque B · función de formato         commit
3º Bloque C · usarla en las pantallas    commit
4º Bloque D · pruebas                    commit
```

Párate después de cada bloque.

CRITERIOS DE ACEPTACIÓN

``` DATOS ☐ Todo monto en datos-falsos es entero ☐ Todo campo de dine